The allegory axioms, and what defines them in the Frobenius calculus

Freyd, *Categories, Allegories* §2.11–§2.112 against `functorialSemanticsForRelationalTheories.pdf` Def. 4.1

Lean: `Freyd/S2_10.lean` (`Allegory`), `diag/CB.lean` (`CartBicat`), `diag/CB_Allegory.lean` (`allegoryOfCartBicat`)

Where the pictures come from. Every picture of a theorem is emitted by `./scripts/diag-export` from that theorem's Lean type. Nothing about a statement is redrawn here. The two figures marked *transcribed* are the exception: they copy the paper's definition pictures, and a definition has no statement to export.

Equation numbering. The numbers below are the 2017 preprint's. The `CartBicat` fields no longer encode them — a field is named for what it says, «∇Δ≤𝟙» rather than `ineq_37` — and each docstring carries the preprint number. The repo's copy of the paper is now the published version — Poly. J. Math. **2** (6), 20 Oct 2025, doi:10.69763/polyjmath.2.6 — which renumbers. Translation, published ← preprint: (34)⟵(37), (35)⟵(38), (36)⟵(39), (37)⟵(40), (38)⟵(41), (39)⟵(42), (40)⟵(43), (33)⟵(35), and Example 2.3(c)'s (13),(14)⟵(11),(12); the preprint's (36) is unnumbered there. Pages: Def. 4.1 at 20–21 (was 17), the † notation at 21 (was 19), Lemma 4.2 at 21–22 (was 19). Renumbering the Lean fields is a separate decision and has not been made.

Freyd's allegory takes reciprocation `°` and intersection `∩` as **primitive operations** and lists eight **equational axioms** on them. A cartesian bicategory of relations takes neither: every object carries one special Frobenius structure `Δ`, `!`, `∇`, `?`, the hom-order `≤` is primitive, and both `°` and `∩` are **defined** from the four dots. Every one of Freyd's eight axioms is then a theorem. The table below is that dictionary, one row per axiom.

Reading the pictures: left to right is composition — the book's diagram order, `R S` means first `R` then `S` — vertical stacking is `⊗`, a bare wire is an identity, a white box is a relation, and a box that is mirrored and tinted is its converse — the mirroring is the paper's own notation, the tint is this note's, because at a glance a chamfer is easy to miss. The label inside is **not** mirrored: the paper can reverse a single letter and still be read, a whole term like `R ∪ S` cannot. All four dots are solid, as Def. 4.1 draws them. The exporter draws a meet or a converse as a **labelled** block, so a composite under a `°` appears as a single box reading `R ; S`; a `⊗` draws as two stacked runs; and any subterm with no shape in this calculus — an associator, say — degrades to an opaque box. The term columns write the converse `R°`, Freyd's symbol, where the paper writes `R†`: these terms are read against the book throughout, and one symbol per idea beats asking the reader to match two. The Lean sources write `R°` as well. The paper reserves `(–)°` for the colour swap of its §7, but §7 is a worked example in other models — see the caution at the end of this note — and nothing here formalises it, so in a repo that is `Rel(Set)` throughout there is no clash to protect against.

In a chain table the symbol at the head of each row is the relation **that step was proved at**: = for a reshaping, ≤ for a genuine inequality. Counting them is the point of drawing a proof at all — a chain of one ≤ between equalities says the theorem is one law and a lot of bookkeeping.

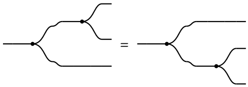
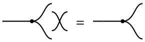
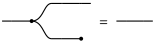
1. Definition 4.1, as the paper states it

`functorialSemanticsForRelationalTheories.pdf` Def. 4.1, pp. 20–21 — transcribed here as `class CartBicat` (`diag/CB.lean`), field for field. Every picture below is exported from the Lean field, so the class and the definition cannot drift apart.

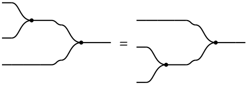
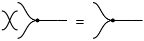
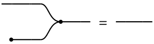
A **cartesian bicategory of relations** is a poset-enriched category, symmetric monoidal, satisfying four conditions. Clauses 1 and 2 are the two structures every object carries; clause 3 makes each the other's adjoint and imposes Frobenius; clause 4 is the only condition on arrows.

The tower this note describes is STRICT: the two bracketings of a threefold product are the same object, so no associator or unitor appears in any picture. The coherence arrows the paper suppresses in its own diagrams are absent here because they do not exist.

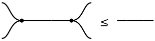
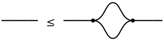
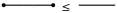
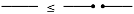
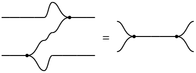
1.1. Every object carries a cocommutative comonoid `(Δ, !)`

	(8) **coassociativity** `CartBicat.Δ_assoc` — copying twice on the left is copying twice on the right.
	(9) **cocommutativity** `CartBicat.Δ_comm` — the two copies are interchangeable.
	(10) **counit** `CartBicat.Δ_counit` — copy, then discard one copy, and nothing has happened.

1.2. Every object carries a commutative monoid `(∇, ?)`

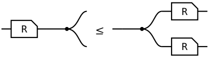
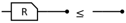
	(5) **associativity** `CartBicat.«∇_assoc»`.
	(6) **commutativity** `CartBicat.«∇_comm»`.
	(7) **unit** `CartBicat.«∇_unit»` — create, then merge, and nothing has happened.

1.3. Five inequations relate the monoid and the comonoid

	(37) `∇ ; Δ ≤ 𝟙` — half of `Δ ⊣ ∇`
	(38) `𝟙 ≤ Δ ; ∇` — the other half
	(39) `? ; ! ≤ 𝟙` — half of `! ⊣ ?`
	(40) `𝟙 ≤ ! ; ?` — the other half
	(41) the **Frobenius** law `CartBicat.frob_left`, and its mirror `frob_right`. This is the one equation of the group; the other four are inequalities.

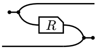
The **special** law `Δ ; ∇ = 𝟙` is NOT part of the definition. The paper derives it — "one direction is given by (38) and the other is proved as follows" — and so does the Lean, as `Freyd.Diag.CartBicat.special`; its picture is in the correspondence table below.

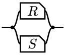
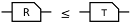
1.4. Every arrow is a lax comonoid homomorphism

	(42) `R ; Δ ≤ Δ ; (R ⊗ R)` `CartBicat.lax_Δ` — running R once and copying is below copying and running it on both strands.
	(43) `R ; ! ≤ !` `CartBicat.lax_!`.

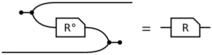
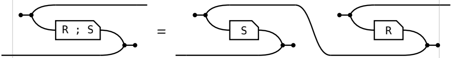
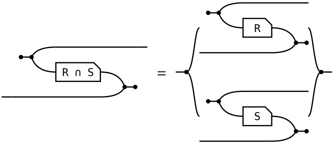
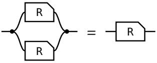
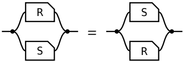
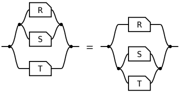
Read as inequalities in one direction only: an arrow that satisfied these as EQUATIONS would be a map, and the whole point is that a general relation is not.

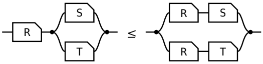
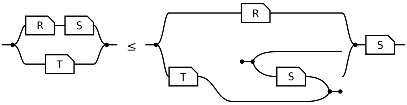
2. Freyd's primitives are the Frobenius calculus's definitions

Freyd's primitive, and what defines it here	picture
reciprocation R° — §2.11, primitive. Here `conv R := bend ((R ⊗ 𝟙) ; cap)` (`diag/CB.lean:223`), not a generator. Input arrives bottom left and runs under R; the **cup** `? ; Δ` at the top left creates a pair, one strand feeding R and the other leaving as the output at the top right; the **cap** `∇ ; !` at the bottom right annihilates R's output against the input. It is the cup and the cap that reverse the arrow — a wire merely bumped up over the box and back down is an isotopy of R itself.	 transcribed

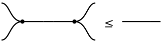
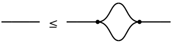
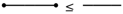
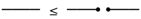
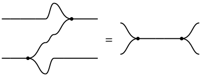
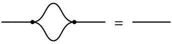
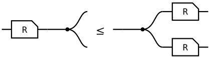
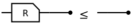
intersection `R ∩ S` — §2.11, primitive. Here the convolution `meet R S := Δ ; (R ⊗ S) ; ∇` (`diag/CB_Derived.lean:24`). Copy the input, run both, merge; the merge forces the two results to agree.	 transcribed
containment `R ⊑ S` — §2.11, **defined** as `R ∩ S = R`. Here the poset-enrichment ≤ is primitive and ∩ is derived, so the two directions are swapped. `meet_eq_left_of_le` (`diag/CB_Derived.lean:196`) is the bridge.	no picture — this is the 2-cell itself
maximal morphism `⊤` — §2.15, and Freyd needs a **unitary** allegory for it. Here `top a b := ! ; ?` (`diag/CB_Derived.lean:28`) is free in Def. 4.1. `Freyd.Diag.«≤_top»`.	

3. The eight axioms, and what proves each

Freyd's axiom, and what supplies it	picture, from the Lean statement
`(R°)° = R` — §2.11 `recip_recip`; here the **snake**, from (41). `Freyd.Diag.CartBicat.conv_conv`. Bending down and back up straightens out, so bending twice is the identity.	
`(R S)° = S° R°` — §2.11 `recip_comp`; here mirroring the picture. `Freyd.Diag.CartBicat.conv_comp`. Reflecting a chain left to right reverses it; each box comes back flipped.	
`(R ∩ S)° = R° ∩ S°` — §2.11 `recip_inter`; the same mirroring. `Freyd.Diag.conv_inter`. Δ and ∇ are each other's mirror image, so reflecting the convolution leaves its shape alone and only turns the boxes round.	
`R ∩ R = R` — §2.11 `inter_idem`; here (42) and the **special** law. `Freyd.Diag.meet_idem`. Copy, run R twice, merge — (42) pushes R back through the copy and `Δ ; ∇ = 𝟙` closes the bubble.	
`R ∩ S = S ∩ R` — §2.11 `inter_comm`; here cocommutativity (9) and commutativity (6). `Freyd.Diag.meet_comm`. The two strands of Δ are interchangeable, and so are those of ∇.	
`R ∩ (S ∩ T) = (R ∩ S) ∩ T` — §2.11 `inter_assoc`; here coassociativity (8) and associativity (5). `Freyd.Diag.meet_assoc`. Either bracketing builds the same three-way copy tree, and dually the same merge tree.	

semi-distributivity `R (S ∩ T) ⊑ R S ∩ R T` — §2.11 `semidistrib`, §2.136; here (42), the **lax comonoid** inequation. `Freyd.Diag.semidistrib_of_lax`. Running `R` and then copying is below copying and then running `R` on each strand.	
modular identity `R S ∩ T ⊑ (R ∩ T S°) S` — §2.112 `modular`, the axiom Freyd **adjoins**: “we adjoin one more containment, the law of modularity”. Here (41) and (42), **derived**. `Freyd.Diag.modular_of_frobenius`. Strip the shared prefix `Δ ; (R ⊗ T)` off both sides and what is left is «`∇_slide_conv`» (`diag/CB.lean:560`) — a box slides through a merge at the cost of its converse on the other strand.	

4. The other direction: what each law of Def. 4.1 says in Freyd’s language

picture, from the Lean statement	Def. 4.1	Freyd
	(37) `∇ ; Δ ≤ 𝟙` half of `Δ ⊣ ∇`	`Δ° Δ ⊑ 1` — **Δ is simple** §2.13. Not used anywhere in the bridge; see below.
	(38) `𝟙 ≤ Δ ; ∇` the other half of `Δ ⊣ ∇`	`1 ⊑ Δ Δ°` — **Δ is entire** §2.13; with (37), Δ is a **map**.
	(39) `? ; ! ≤ 𝟙` half of `! ⊣ ?`	`!° ! ⊑ 1` — **! is simple** §2.13. Also unused in the bridge.
	(40) `𝟙 ≤ ! ; ?` the other half of `! ⊣ ?`	`1 ⊑ ! !°` — **! is entire** §2.13; with (39), `!` is a map. This is what makes `⊤ = ! ; ?` contain every arrow.
	(41) the **Frobenius** law an axiom. `𝟙 ⊗ Δ` and `∇ ⊗ 𝟙` draw as stacked runs; only the associator `α⁻¹` stays an opaque box, having no shape in this calculus.	the **modular identity** §2.112 — and reciprocation §2.11 itself, since the cup and cap only satisfy the snake because of (41).
	the **special** law **not** an axiom: derived from (38) and (40), `Freyd.Diag.CartBicat.special`	`Δ Δ° = 1` — **Δ is injective**, so by §2.141 at `f = g = Δ` it is a monic map. In `Rel(Set)`: copying is total and loses nothing.
	(42) every arrow is a **lax comonoid homomorphism**	§2.136′s `R (S ∩ T) ⊑ R S ∩ R T`, read at `S := π₁°`, `T := π₂°` — Freyd’s semi-distributivity, product-free.
	(43) every arrow is **lax for the counit**	§2.152′s step “`p_α` is maximal in `(α, λ)`, hence `R p_β ⊆ p_α`” — that is, `R ⊑ ⊤`.

5. Unitary and pre-tabular — the two conditions the converse needs

Freyd’s §2.11 axioms are all a cartesian bicategory has to give back. Going the other way needs two further conditions, and they are exactly the two the repo has not formalised. Both definitions below are quoted from `Freyd/S2_10.lean`, which follows the book.

condition	Freyd	what the Frobenius side already has
unitary §2.15	`T` is a **partial unit** when `𝟙 T` is the largest endomorphism on it, `PartialUnit T := ∀ R : T → T, R ⊑ 𝟙 T` (`:539`); it is a **unit** when moreover every object carries an entire arrow to it, `IsUnit T := PartialUnit T ∧ ∀ a, ∃ R : a → T, Entire R` (`:545`). A `UnitaryAllegory` is an allegory with one (`:548`). The point of a unit is that it makes `T_{a,b}` exist: `⊤ = p_α p_β°` through the unit, which is where §2.152′s “`R p_β ⊆ p_α`” — the repo’s (43) — comes from.	**already there, for free.** The monoidal unit 𝕀 is the unit object, `!_a : a → 𝕀` is the entire arrow every object carries to it — (40) says so — and `⊤ = ! ; ?` is maximal by `Freyd.Diag.«≤_top»`. So every cartesian bicategory of relations is unitary, with nothing assumed. Not recorded in Lean: `allegoryOfCartBicat` builds a plain `Allegory`, and no `UnitaryAllegory` instance is derived from `CartBicat`.

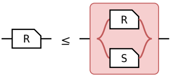
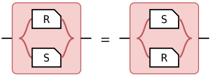
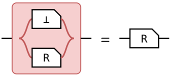
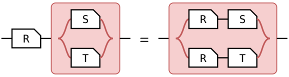
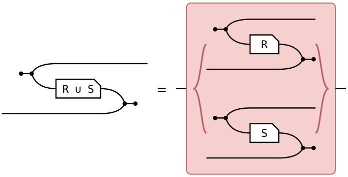
pre-tabular §2.165	`f, g` **tabulate** `R` when both are maps, `R = f° g`, and `f f° ∩ g g° = 𝟙` (`Tabulates`, `:502`); `R` is **tabular** when some pair tabulates it (`:506`). An allegory is `PreTabularAllegory` when every morphism is contained in a tabular one (`:556`). §2.165: “If A is unitary then pre-tabularity is equivalent to the existence of tabulations for maximal morphisms.”	**this is what ⊗ is.** Given unitarity, the condition reduces to tabulating each `T_{a,b}`, and a tabulation of `T_{a,b}` is precisely an object `a ⊗ b` with its two projections — the monoidal product Def. 4.1 assumes. That is the whole content of the swap: Freyd derives the product from tabulations, Def. 4.1 posits it. Not built. Rebuilding ⊗ from tabulations and products in `Map(𝒜)` is `cartBicatOfUnitaryPretabular`; `diag/PLAN.md` holds it back for want of a consumer.

Together they are the whole gap: **cartesian bicategory of relations ≃ unitary pre-tabular allegory** (`DiagrammaticAlgebraOfFirstOrderLogic` §10, after Carboni–Walters). One half of that equivalence is `allegoryOfCartBicat` and is a theorem in this repo; the other half is the two rows above.

6. Union and residual — what phases 8 and 9 add

`∪` and `/` are past where Freyd’s §2.11 list stops, and past what Def. 4.1 can build: they need a **biproduct** on top of the Frobenius structure (`diag/Tape.lean`, after `TapeDiagrams.pdf`) and a **second composition** with linear adjoints (`diag/FO.lean`, after `DiagrammaticAlgebraOfFirstOrderLogic.pdf`). Both are built, so both draw.

The two notations say which layer a picture is in. A union is a **tape**: the rounded wrapper is the second monoidal product, and its ▷/◁ open and close a branch a particle takes exactly one of — the same fork-runs-join shape as a meet, on a different product. A residual is a **cut**: `R / S` is `R ;• S⊥` by definition, the black composition is drawn on black ground and the linear adjoint `S⊥` is `S` mirrored and negated, so the picture is `R` on black beside `S` mirrored in a white island. Both conventions are `DiagrammaticAlgebraOfFirstOrderLogic.pdf`’s own, and it states the second one twice: on p. 2, given `c`, “take its mirror image and then its photographic negative” — that is `c⊥`; and on p. 10, translating Peirce’s existential graphs, “each cut [is] a **color switch**”. Its Fig. 4 draws the black composition the same way, as a black ground, which is what (δl) and (δr) are pictures of. Nesting alternates the ground, so the white island inside a black region is Peirce’s double cut.

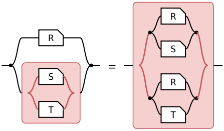
statement	**picture, from the Lean statement**
`R ⊑ R ∪ S` — §2.2 in Freyd; here `Freyd.Diag.Biprod.«≤_union_left»`. `union R S := ⟨R, S⟩ ; [𝟙, 𝟙]` (`diag/Tape.lean:84`): offer both branches, then forget which was taken.	
`R ∪ S = S ∪ R` — `Freyd.Diag.Biprod.union_comm`. The two branches of a tape are interchangeable, exactly as the two strands of Δ are.	
`⊥ ∪ R = R` — `Freyd.Diag.Biprod.bot_union`. `bot a b := toZero a ; (toZero b)°` (`diag/Tape.lean:87`), routed through the zero object; the exporter has no shape for it, so it prints as a box.	
`R (S ∪ T) = R S ∪ R T` — `Freyd.Diag.Biprod.comp_union`; composition distributes over union, which `∩` does not. Freyd §2.2′s distributivity, the reason a distributive allegory is more than an allegory.	
`(R ∪ S)° = R° ∪ S°` — `Freyd.Diag.Biprod.conv_union`. The converse is a 2-functor for the tape layer too.	

<table>
<tr>
<td>

`R ∩ (S ∪ T) = (R ∩ S) ∪ (R ∩ T)` — `Freyd.Diag.FbCbRig.meet_union_distrib`, the law that makes `distributiveAllegoryOfFbCb` a **distributive** allegory (§2.2).

The one picture with both layers in it: meets inside a tape on the left, tapes around meets on the right.

</td>
<td>

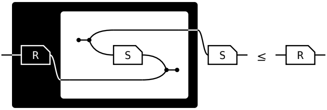


</td>
</tr>
<tr>
<td>

`(R / S) S ⊑ R` — `Freyd.Diag.ClosedLinearBicat.«residual_comp_≤»`; the residual is a lower bound of the arrows it is the greatest of.

Freyd's §2.31 division: `R / S` is the largest `T` with `T S ⊑ R`, and `«≤_residual_iff»` (`diag/FO.lean:171`) is that Galois statement in full. The two colour switches are the definition `R ;• S⊥` unfolded — one for the black composition, one for the linear adjoint — and the `S` in the island is mirrored because a linear adjoint is a converse as well as a negative.

</td>
<td>



</td>
</tr>
</table>

7. Lemma 4.2 (ii) — the proof, not just the statement

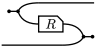
`diag-export --proof` draws a **proof**. A `calc` chain elaborates to nested `Trans.trans`, so its intermediate terms are all recoverable from the proof term: flatten the spine, infer the type of each leaf, draw one canvas per step. Nothing is replayed and nothing is retold — the nine rows below are the chain the Lean kernel accepted for `Freyd.Diag.CartBicat.conv_comp`, which is Lemma 4.2 (ii), `(R ; S)° = S° R°`.

7.1. First: why these pictures are not pictures of a converse

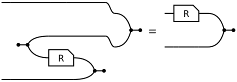
They are not, and that is the thing to get straight before reading them. The converse is drawn with **two** bends — a cup on the way in and a cap on the way out, each a pair of dots, `? ; Δ` and `∇ ; !` — which is the left-hand figure below. The proof never draws that. `conv_unique` says an arrow is **the** converse of `R` as soon as it satisfies one equation between UNBENT arrows `a ⊗ b → I`: two strands in on the left, one cap on the right. Bending is a bijection (`bend_unbend`/`unbend_bend`), so settling the unbent equation settles the bent one, and the cup never has to be drawn at all.

That is why every row below has a cap and no cup, and why an arrow that "is a converse" shows up not as a bent box but as a box sitting on the **bottom** strand, mirrored.

<table>
<tr>
<th>the converse, as the paper draws it</th>
<th>the one rule the proof uses, twice</th>
</tr>
<tr>
<td>



</td>
<td>



</td>
</tr>
<tr>
<td>

`R°` is `R` with both wires bent round: in at the bottom left, cup at the top left, cap at the bottom right. Two dots at each bend, as `functorialSemanticsForRelationalTheories.pdf` draws it — a cup is `?` then `Δ`, a cap is `∇` then `!`. **Transcribed by hand;** a definition has no statement to export.

</td>
<td>

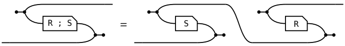
`conv_slide` (`diag/CB.lean:449`): a box on the BOTTOM strand facing a cap is the same as that box on the TOP strand, upright. Sliding round the bend is what turns `R°` into `R`. This is the entire content of Lemma 4.2 (ii); the other seven steps are bookkeeping.

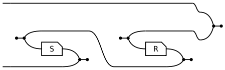
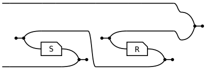
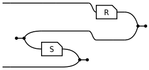
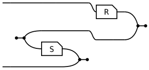
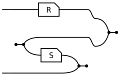
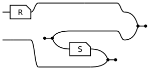
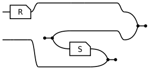
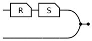
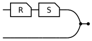
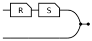
</td>
</tr>
</table>

7.2. The chain

One row per **term**, not per step: the right-hand side of every step is the left-hand side of the next, so drawing steps would draw every picture twice. The first column is the term as the Lean has it, fully bracketed, with `⊗` binding tighter than `;` — it comes off the same walk as the picture, so the two cannot drift apart. Watch which **strand** each box is on: bottom means mirrored, top means upright.

Lemma 4.2 (ii), `(R ; S)° = S° ; R°`



term	picture	the rule that reaches it
`𝟙 ⊗ (S° ; R°) ; cap`		**the start.** What `conv_unique` asks about `S° R°`: both boxes on the bottom strand, mirrored — that is what “these are converses” looks like once the wires are unbent.
`= 𝟙 ⊗ S° ; (𝟙 ⊗ R° ; cap)`	= 	**1.** ⊗ is a functor, so doing two things to one strand in sequence is the sequence of doing them. **Same picture.**
`= 𝟙 ⊗ S° ; (R ⊗ 𝟙 ; cap)`	= 	**2. `conv_slide`.** `R°` is the box nearest the cap, so it slides round the bend: it leaves the bottom strand, arrives on the top, and comes back upright as `R`. **The first of the two steps that prove anything.**
`= (𝟙 ⊗ S° ; R ⊗ 𝟙) ; cap`	= 	**3.** `simp only [Cat.assoc]`. Re-bracketing — visible in the term column, invisible in the picture. **Same picture.**
`= R ⊗ S° ; cap`	= 	**4. `tensHom_split'`** — interchange. `R` is on the top strand and `S°` on the bottom, so running them in sequence and running them side by side are the same thing. Shows as a re-spacing only because the exporter lays a diagram out left to right.
`= (R ⊗ 𝟙 ; 𝟙 ⊗ S°) ; cap`	= 	**5. `tensHom_split`** — the same law, back the other way. Split them apart again, `R` first, so that `S°` is now the box facing the cap. Which is what the next real step needs.
`= R ⊗ 𝟙 ; (𝟙 ⊗ S° ; cap)`	= 	**6.** `simp only [Cat.assoc]`. Re-bracketing. **Same picture.**
`= R ⊗ 𝟙 ; (S ⊗ 𝟙 ; cap)`	= 	**7. `conv_slide`,** on `S` this time. `S°` slides round the bend and arrives upright on the top strand. Both boxes are now on top, both upright, in the order `R` then `S`. **The second and last real step.**
`= (R ⊗ 𝟙 ; S ⊗ 𝟙) ; cap`	= 	**8.** `simp only [Cat.assoc]`. Re-bracketing. **Same picture.**
`= (R ; S) ⊗ 𝟙 ; cap`	= 	**9. `tensHom_comp`** — interchange once more; the two top-strand boxes merge into one. This is `R ; S` unbent, so `conv_unique` reads off `S° R° = (R ; S)°`. **Done.**

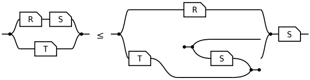
What the chain measures. Read down the pictures and most consecutive pairs are the same drawing — steps 1, 3, 6, 8 and 9 change nothing at all, and step 4 changes only spacing — while the term column beside them changes at every row. Those are associativity, the unit laws and interchange — precisely the laws a string diagram has built into its geometry rather than as rewrites. Seven of these nine Lean steps are bureaucracy the picture language does not charge for, and the theorem is two applications of `conv_slide`.

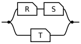
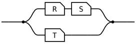
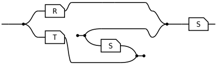
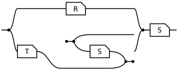
How this differs from the paper's proof. The paper proves (ii) by inserting a snake in the middle of `R ; S` and reading the two halves off as `S°` and `R°` — three pictures, all bent. The Lean never bends. Same theorem, different route, and the export shows the route the kernel accepted. Drawn by hand it would have shown the paper's three pictures and quietly implied those were what the repo had checked.

8. The modular law — proved, drawn

Freyd **adjoins** the modular identity: an allegory has no structure to derive it from, so it goes in as an axiom. Here it is a theorem, and this is its proof — `Freyd.Diag.modular_of_frobenius` (`diag/CB_Allegory.lean:48`), four terms, of which exactly one step is an inequality.

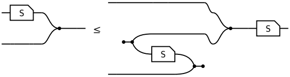
the modular law, `R S ∩ T ≤ (R ∩ T S°) S`



term	**picture**	**the rule that reaches it**
`(R ; S) ∩ T`		**the start:** `R S ∩ T`. A meet, so a copy and a merge with `R S` above and `T` below.
`= (Δ ; R ⊗ T) ; (S ⊗ 𝟙 ; ∇)`	= 	**Factor out the shared prefix.** `(R S) ⊗ T` is `(R ⊗ T) ; (S ⊗ 𝟙)`, so everything before the `S` is common to both sides and can be set aside. An equality, and drawn as one.
`≤ (Δ ; R ⊗ T) ; (𝟙 ⊗ S° ; (∇ ; S))`	≤ 	**«`∇_slide_conv`»,** under the shared prefix. `S` slides through the merge and reappears as `S°` on the other strand. **The only inequality in the proof** — everything else is reshaping.
`= R ∩ (T ; S°) ; S`	= 	**Fold the prefix back in.** `R ⊗ (T S°)` is `(R ⊗ T) ; (𝟙 ⊗ S°)`, and the copy-merge pair reads as a meet again. `(R ∩ T S°) S`, which is the goal.

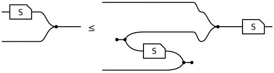
8.1. Inside the middle step

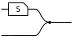
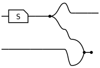
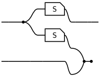
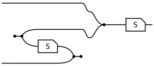
«`∇_slide_conv`» (`diag/CB.lean:560`) is where the mathematics is, and it mentions only `S` — `R` and `T` sit untouched in the prefix throughout. Read on its own it says one arrow slides through a merge:



`S` occurs once on the left and twice on the right. That is the tell: the lax copy inequation (42) is the only law in Def. 4.1 that may duplicate a box, so it has to be the inequality step — and its own chain shows it is the only one.

«`∇_slide_conv`», `(S ⊗ 𝟙) ; ∇ ≤ (𝟙 ⊗ S°) ; ∇ ; S`

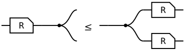
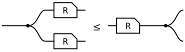
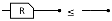
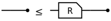
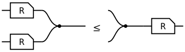
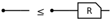


term	picture	the rule that reaches it
`S ⊗ 𝟙 ; ∇`		**the start:** `(S ⊗ 𝟙) ; ∇`.
`= (S ; Δ) ⊗ 𝟙 ; 𝟙 ⊗ cap`	= 	**«`∇_of_cap`»,** backwards. The merge is rebuilt from a copy and a cap, which puts the term into the shape `(S ; Δ) ⊗ 𝟙` that (42) applies to.
`≤ (Δ ; S ⊗ S) ⊗ 𝟙 ; 𝟙 ⊗ cap`	≤ 	**(42), `lax_Δ`.** `S ; Δ` becomes `Δ ; (S ⊗ S)`: the box is copied. **The whole of the modular law, in one step.**
`= 𝟙 ⊗ S° ; (∇ ; S)`	= 	**«`cap_tens_∇`» and `conv_slide`.** Reshaping back, and the surviving duplicate slides round the cap to become `S°` — the same rule Lemma 4.2 (ii) is made of.

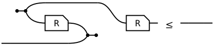
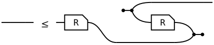
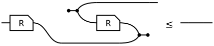
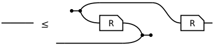
What the associators cost. α and ρ have no shape in this calculus and print as opaque boxes, so the two middle terms of the inner chain are less legible than its ends. That is the honest price of drawing what the Lean says: the reshaping steps really are that fiddly, and a hand-drawn version would have hidden them by working up to coherence.

9. Maps: SV, TOT, INJ, SUR, and the two inequations that always hold

Def. 4.1 clause 4 says every arrow is lax for `Δ` and for `!` — inequations (42) and (43). Applying Lemma 4.2 (ii) and (iv) to them gives (44) and (45), the same statements about `∇` and `?`. Those four hold for **every** arrow. Their four REVERSES do not, and each one failing or holding is a property of the arrow: that is (SV), (TOT), (INJ), (SUR).

	holds for every arrow	holds iff the arrow is …
copy `Δ`	(42) `R ; Δ ≤ Δ ; (R ⊗ R)` 	(SV) — **single valued** 
discard `!`	(43) `R ; ! ≤ !` 	(TOT) — **total** 
merge `∇`	(44) `∇ ; R ≤ (R ⊗ R) ; ∇` — the converse of (42). **Not in the Lean;** see below.	(INJ) — **injective** 
unit `?`	(45) `? ; R ≤ ?` — the converse of (43). **Not in the Lean;** see below.	(SUR) — **surjective** 

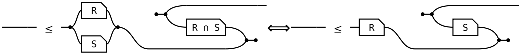
The paper gives the same four properties a second time, as ADJOINT conditions — (46)–(49) — and Lemma 4.4 says the two presentations agree. `diag/CB_Derived.lean` takes the adjoint forms as the definitions, because they are literally Freyd's `Simple` and `Entire` (§2.13), so a `diag` map and a book map are one predicate with no translation layer.

property	adjoint form, the definition	Freyd
single valued `Freyd.Diag.SingleValued`		`Simple R := R° R ⊑ 1` §2.13
total `Freyd.Diag.Total`		`Entire R := 1 ⊑ R R°` §2.13. With **single valued**, `R` is a **map**.
injective `Freyd.Diag.Injective`		`R R° ⊑ 1` §2.141 at `f = g = R`: a monic pair, so a monic map.
surjective `Freyd.Diag.Surjective`		`1 ⊑ R° R` §2.147: “`g` is a cover iff `1 ⊑ g° g`”, that is, iff `g°` is entire.

Why (44) and (45) have no picture. Both are converse-images, so drawing them from a Lean statement needs that statement to exist, and it does not. Getting it needs `Δ° = ∇`, `!° = ?` and Lemma 4.2 (iii) `(f ⊗ g)° = f° ⊗ g°` — and the first of those is a Frobenius computation at a COMPOSITE object, which is the clause of Carboni & Walters' definition that Def. 4.1 as printed omits (`diag/CB.lean`'s header records this). It is the same gap that leaves Lemma 4.4 unproved in the repo. The four reverse inequations opposite are unaffected: they are recorded under the paper's own labels in `diag/CB_Derived.lean`, and nothing downstream depends on Lemma 4.4.

10. Entireness of an intersection

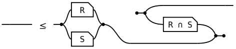
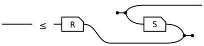
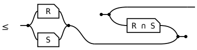
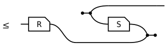
Two definitions from Freyd, both used below. The **domain** of `R` is the part of the identity where `R` is defined, `Dom R = 1 ∩ R R°` (§2.122). `R` is **entire** if it is defined everywhere, `Dom R = 1`, equivalently `1 ⊑ R R°` (§2.13) — which is `Total` above, the picture opposite (47). The question this section answers is when an **intersection** is entire: Freyd's §2.124, "a lemma we will use repeatedly", and B&dM 4.18.

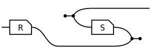
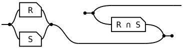


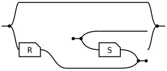
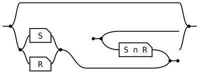
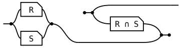
`Freyd.Diag.entire_inter_iff` (`diag/S2_124.lean:213`) — `Total (R ∩ S) ↔ 𝟙 ≤ R ; S°`

Read the two sides: on the left `R ∩ S` is named **twice**, on the right `R` and `S` are named **once each** and the meet is gone. That is the whole point of the lemma — the meet is needed to **ask** whether something is entire, not to **answer** it — and it is why every later use quotes the right-hand form. Freyd spends it at §2.147, where `H = x f° ∩ y g°` has to be shown entire; B&dM spend it throughout chapter 4.

`Freyd.Alg.entire_inter_iff` (`AOP/A4_2.lean:187`) is the same statement for a bare allegory, and there it is `Iff.rfl` on top of `dom_inter`: in `Freyd/S2_10.lean` the order `X ⊑ Y` is **defined** as `X ∩ Y = X`, so "`Dom(R ∩ S) = 1`" and "`1 ⊑ R S°`" are the same proposition once §2.124 is known. Here ≤ is primitive, so both directions are real steps — and they cost very different things.

⟹ given `R ∩ S` entire,  show 		
term	**picture**	**the rule that reaches it**
`𝟙`		**the start:** `𝟙`.
`≤ R ∩ S ; (R ∩ S)°`		**the hypothesis:** `R ∩ S` entire. By the definition of entire, `𝟙 ≤ P P°` at `P = R ∩ S`.
`≤ R ; S°`		**monotonicity, twice:** `R ∩ S ≤ R`, and `R ∩ S ≤ S` under `°`. No modular law.

⟸ given —— ≤  show `R ∩ S` entire, —— ≤ 

term	picture	the rule that reaches it
`𝟙`	——	**the start:** `𝟙`.
`≤ 𝟙 ∩ (R ; S°)`	≤ 	**the hypothesis,** met with `𝟙`. `meet_glb`: `𝟙 ≤ 𝟙` and `𝟙 ≤ R S°`, so `𝟙` is below the meet of the two.
`= 𝟙 ∩ (S ∩ R ; (S ∩ R)°)`	= 	**§2.124:** `𝟙 ∩ R S° = 𝟙 ∩ (S ∩ R)(S ∩ R)°`. `R S°` relates `a` to `a′` when some `b` has `a R b` and `a′ S b`. The `𝟙 ∩` sets `a′ = a`, and it then reads “some `b` has `a R b` and `a S b`” — one arrow of `R ∩ S`. Two arrows become one; that is the step, and it is what needs the modular law. Freyd writes the right-hand side `Dom (S ∩ R)`: `Dom X` is his name for `𝟙 ∩ X X°`, so it is the same term, not an extra operation.
`≤ R ∩ S ; (R ∩ S)°`	≤ 	**`X ∩ Y ≤ Y`,** then `S ∩ R = R ∩ S`. The outer fork drops away, leaving the goal.

What the asymmetry says. One direction is two applications of monotonicity; the other needs §2.124, which needs the modular law, which in this calculus needs the Frobenius equation (41) and the lax copy inequation (42) — §7 above. So the picture chain locates the cost exactly: the third row of the second table is the whole strength of the lemma, and everything else is bookkeeping.

11. Anything missing?

Nothing on Freyd's side. All eight §2.11 axioms are theorems of Def. 4.1, and `allegoryOfCartBicat` (`diag/CB_Allegory.lean:85`) discharges the `Allegory` structure field for field. The modular identity, which Freyd has to **adjoin** because an allegory has no structure to derive it from, comes out of (41) and (42).

Three things are stated but unproved. «`∇Δ≤𝟙`» and «`?!≤𝟙`» are discharged and never used (see below); (44) and (45) are not stated at all; and Lemma 4.4, that the comonoid and adjoint forms of the map conditions agree, is not proved. All three trace to one omission in Def. 4.1 as printed — no Frobenius structure at a composite object — which `diag/CB.lean`'s header records.

Two laws on the Frobenius side do no work here. Grepping the repo, «`∇Δ≤𝟙`» and «`?!≤𝟙`» are never used: they appear only in the class definition. They are there to make `Δ ⊣ ∇` and `! ⊣ ?` genuine adjunctions — which is what pins `∇ = Δ°` and `? = !°` by uniqueness of adjoints — not to prove any allegory law.

And nothing on this branch discharges them, because the tower has no model. `diag/RelSetCB.lean` and `diag/RelSetFO.lean` carried the `Rel(Set)` instances and were dropped when the monoidal structure was made strict: `RelSet` is not a model of `diag/Monoidal.lean`, so, as `diag/FO.lean`'s own header puts it, “the layer has no model here”. Everything below Def. 4.1 in this note is a theorem OF the axioms; that the axioms are satisfied by relations is the paper's, not this repo's.

The bridge runs one way only, for the reason the section above sets out: a cartesian bicategory carries a symmetric monoidal ⊗ and an allegory carries nothing of the kind.

What Def. 4.1 alone does not have. Everything past Freyd's §2.11 needs structure beyond it, and — this is the point — from **different sources**, not from `functorialSemanticsForRelationalTheories.pdf`:

construct	Freyd	diagrammatic counterpart	status
tabulation	§2.14	none	no counterpart. Def. 4.1 gives no splitting of coreflexives; `⊤ = ! ; ?` is all the tabular content there is.
`∪`, `⊥`	§2.2, distributive allegory	a **biproduct** ⊕ on top of Def. 4.1 — `Biprod`, `union`, `bot`, and `distributiveAllegoryOfFbCb` in `diag/Tape.lean`; `TapeDiagrams.pdf`	built, and drawn above. Not derivable from Def. 4.1 alone — the paper's calculus has no union at all.
`R / S`, complement	§2.31, division allegory	`LinearBicat`, `ClosedLinearBicat`, `residual` — `diag/FO.lean`; `DiagrammaticAlgebraOfFirstOrderLogic.pdf`	built, and drawn above — but axiomatised only. `perp` is a field of `ClosedLinearBicat` that nothing on this branch instantiates.
`Λ`, power transpose	§2.4	none	open. No calculus in this tower has a power object.
Allegory ⟹ CB	§2.15 + §2.165	`cartBicatOfUnitaryPretabular`	deliberately not built.

One caution on §7–§9 of the paper, which look like they extend this table and do not: they are worked examples in **other** models. The paper says so itself at the end of §7 — only the black fragment `⊤, ∩, (−)†, ; , id` coincides with the calculus of relations there, while "`⊥` is not the empty relation and > is not the union".

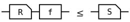
12. The shunting rule

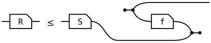
A map may be moved from one side of a containment to the other, at the cost of its converse. That is the workhorse of relational program calculation — B&dM 4.19 and 4.20 — and it is what makes point-free derivations possible at all: it is how a function gets out of the way so the relation underneath can be reasoned about.

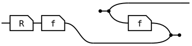
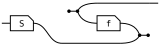
`AOP/A4_2.lean` has both rules for a bare `Allegory` (`map_shunt_right`, `map_shunt_left`), which is where every downstream derivation calls them. They are now also in the diagrammatic layer (`shunt_right`, `shunt_left` in `diag/CB_Derived.lean`) — same statements, so that there is a Lean declaration in `diag` terms for the exporter to draw. Nothing new is assumed: both are proved from `Map` alone.

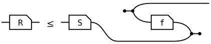
Each rule is an ↔, so each is two chains. The statement of a branch is its header: the hypothesis it is given and the goal it reaches, both drawn from the same Lean ↔ the chain proves.

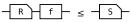
The proofs, drawn. Each ↔ is two chains, and `diag-export --proof` now splits an `Iff.intro` into its two branches rather than drawing the longer one and dropping the other. The point to watch is that each direction spends exactly one half of `Map` — never both.

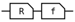
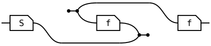
shunting right, ⟹ given 

show 

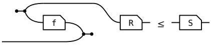
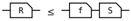
term	picture	the rule that reaches it
`R`		**the start:** `R`.
`≤ R ; (f ; f°)`	≤ 	**`Total f.`** `𝟙 ≤ f f°`, so `f f°` may be inserted after `R`. This is the only thing the forward direction costs.
`≤ S ; f°`	≤ 	Re-bracket to `(R f) f°` and apply the hypothesis `R f ≤ S`. `R ≤ S f°`, which is the goal.

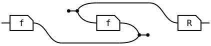
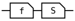
shunting right, ⟸ given 

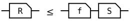
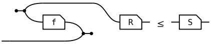
show 

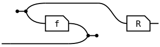
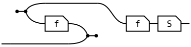
term	picture	the rule that reaches it
`R ; f`		**the start:** `R f`.
`≤ S ; (f° ; f)`	≤ 	Apply the hypothesis `R ≤ S f°` on the left. The two `f`s are now adjacent.
`≤ S`	≤	**`SingleValued f`.** `f° f ≤ 𝟙`, so the pair cancels and `S` is left. The only thing the backward direction costs.

Shunting left is the mirror, and its two chains come out the same shape — `Total f` on the way out, `SingleValued f` on the way back — with the map on the other side throughout.

shunting left, ⟹ given  show 

term	picture	the rule that reaches it
`R`		**the start:** `R`.
`≤ f ; (f° ; R)`	≤ 	**`Total f`.** `f f°` inserted before `R`.
`≤ f ; S`	≤ 	Re-bracket to `f (f° R)` and apply the hypothesis. `R ≤ f S`.

shunting left, ⟸ given  show 

term	picture	the rule that reaches it
`f° ; R`		**the start:** `f° R`.
`≤ (f° ; f) ; S`	≤ 	Apply the hypothesis on the right. The two `f`s become adjacent.
`≤ S`	≤	**`SingleValued f`.** The pair cancels, leaving `S`.

What the rule really is. An adjunction. `R f ≤ S ⟺ R ≤ S f°` says exactly that `f ⊣ f°` in the poset-enriched sense of (35) — `Total f` is its unit and `SingleValued f` its counit — so "`f` is a map" and "`f` has a right adjoint, namely `f°`" are the same statement. That is Lemma 4.8 (`Freyd.Diag.lemma_4_8`), which turns it round: any right adjoint of a map **must** be its converse, by the usual uniqueness argument. The paper's converse half — an arrow with a right adjoint is a map — is the diagram chase that needs Lemma 4.4, and is not proved here for the reason given above.

Where it is spent. In this repo, `map_shunt_right`/`map_shunt_left` carry the relational-product universal property (`AOP/A5_2.lean:177`), division (`AOP/A4_4.lean:370`), the lax naturality conditions (`AOP/A5_7.lean:67`) and the catamorphism work (`AOP/A6_5.lean:372`). It is the single most-used lemma of the AOP layer.

13. Division — half of the cheatsheet's §8, and the half that matters

`AOP/Cheatsheet.typ`'s eighth section is B&dM §4.4–4.5, division and negation. Only division is drawn here, and the reason is not layout.

Negation is not how the book specifies programs. Grepping the formalisation: `~` and `BooleanAllegory` occur in four files of `AOP/`, and every one of them is theory. §4.5 itself; the complement of a **coreflexive**, `~X ∩ 𝟙`, so that a conditional can split on a guard (`AOP/A5_3.lean:188`); one direction of "well-founded = inductive" (`AOP/A6_5.lean:203`), the other direction needing nothing; and subtraction. Chapters 7 to 10, where the programs are actually specified and derived — greedy, thinning, dynamic programming, Horner, knapsack, paragraph formatting, string edit, bracketing, compression — are twenty-four files with **zero** occurrences, working under `UnguardedPowerLCDA` and `TabularUnitaryDivisionAllegory`: locally complete, division, power objects, not Boolean. One thing in those chapters looks like negation and is not: `mnl R ≜ min(R° ⇒ R)` (`AOP/A7_1.lean:432`) uses **implication**, a `Sup` that lives in any locally complete allegory. `~R` is its special case `R ⇒ ∅`, and the book only takes it in §4.5.

So the specification vocabulary is `;`, `∩`, `∪`, `°`, `/`, `Λ`, `min`/`max`, `thin` and catamorphisms, and a note about what those pictures look like has no business spending a page on the one operation the book keeps at arm's length.

Which also settles which picture of `/` is the right one. There are two. `residual` is `R ;• S⊥` and draws as a cut — that is the section above — but `S⊥` is `~(S°)`, so the picture needs a complement, and the paper's Prop. 6.5 makes every homset of a first order bicategory a Boolean algebra. Division asks for nothing of the kind. A `DivisionAllegory` (`Freyd/S2_30.lean:30`) wants one thing, the Galois connection `T ⊑ R/S ⟺ T S ⊑ R`, and `Rel(Set)` satisfies it with a bare ∀: `(R / S) x y ≜ ∀z. S y z → R x z` (`AOP/A6_1_RelSet.lean:157`), no negation in the definition. `Rel` over a non-Boolean topos is a division allegory with no ~ to draw with at all. What follows is the account that needs none.

What stays a box. Dashed means "no definition to unfold and no metaphor either", which here leaves one thing: `/ₛ`, a meet of two converses. A dashed box takes a label, so whatever sits under one is set as text — `(R /ₛ S)°` is a mirrored box reading `R /ₛ S`, not a picture of the meet in its definition. Maps are the other exception, the other way round: `f` is a plain **rectangle**, since the chamfer exists to say which way a relation runs and a map runs one way by construction.

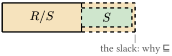
The residual drawn earlier is this `/`. «`≤_residual_iff`» (`diag/FO.lean:171`) and `le_div_iff` (`Freyd/S2_30.lean:54`) are the same Galois statement, `T S ⊑ R ⟺ T ⊑ R/S`, so in a model carrying both structures the two operations agree by uniqueness of adjoints, with neither definition being unfolded.

What `/` is, without a complement anywhere: long division. `R / S` is how much of `R` you can lay down before `S` still fits inside what is left, and the figure says exactly that. Read the bar left to right, as a composite: `R / S` first, then `S`, and the two together stay inside `R`. The leftover is the whole reason the cancel law is `(R/S) S ⊑ R` and not an equality — and it is drawn as a hairline on purpose, because `R / S` is the **largest** such quotient. Any wider and `S` stops fitting; the sliver is what is left when nothing more can be taken.

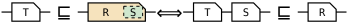
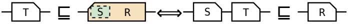
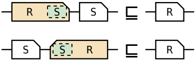
This figure is also the `/` box in every picture below. Where the exported diagrams used to carry a dashed box with `R / S` printed in it — a frame around the same string the term column was already showing — they now carry this bar in miniature: the amber ground is the numerator, the dashed green tile is the divisor laid inside it, and the chamfered corner is the ordinary one every relation box has, so it wires up and says which way it runs. Left division mirrors both, chamfer and tile together, because `S \ R` lays `S` down first.

Solid is what the equation pins: `R`'s left edge, `S`'s left edge, and the stretch before `S`, which is what `R / S` measures. Dashed is the slack: nothing to the right of `S` is determined by `T S ⊑ R`, so `S`'s far edge, `R`'s far edge and the gap between them are all drawn as not-pinned.

Read it as a **measurement**, not as a containment of relations. `R : a → c` and `S : b → c` are not in the same hom-set, so `S` is not a sub-relation of `R` and neither is `R / S` — what sits inside `R` is the composite `(R/S) S`, which does have `R`'s type. The bar is that composite, and the quotient is how far along it reaches. Pushed too far, the metaphor breaks in the usual place: `⊥ / ⊥ = ⊤`, which no picture of a part of `⊥` is going to show.

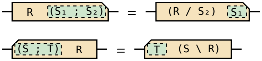


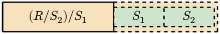
transcribed — the universal property has a Lean statement, this metaphor does not

the ten laws, their exported pictures, and — where the metaphor reaches — the same law as long division
Division is the right adjoint of composition: `T ⊑ R/S ⟺ T S ⊑ R` 
`T ⊑ S\R ⟺ S T ⊑ R` `S\R ≜ (R°/S°)°`: left division is right division conjugated by the converse, not a second primitive. 
cancel: `(R/S) S ⊑ R`, and `S (S\R) ⊑ R` The counit of each adjunction: the quotient composed back with the divisor lands under the numerator, never on it. This is the law the figure above the table draws, so it is not drawn again here. 

associate: `R/(S₁ S₂) = (R/S₂)/S₁`, and `(S T)\R = T\(S\R)`

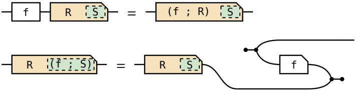
Dividing by a composite one factor at a time. The composition is drawn on the left of each equation and swallowed into a label on the right — and the bar shows what that swallowing hides: two tiles laid where there was one.





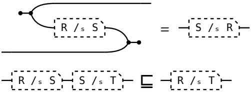
maps: `f (R/S) = (f R)/S`, and `R/(f S) = (R/S) f°`

`f` is a rectangle, so the first law reads off the picture: the same three pieces, bracketed two ways. That is exactly the licence to drop the brackets and write `f R / S`. The second is the one that pays — a map moves out of a denominator and reappears as `f°` outside the box. Both are the shunting rule of the previous section, spent twice. No bar: the metaphor has no handle on prefixing by a map.



symmetric division: `(R/ₛS)° = S/ₛR`, and `(R/ₛS)(S/ₛT) ⊑ R/ₛT`

`R/ₛS ≜ (R/S) ∩ (S/R)°`. The meet is inside the definition, not inside the statement, so it stays under the label: what the pictures show is that `/ₛ` is converse-symmetric and transitive, i.e. behaves like an equality of columns. No bar: a meet of two converses is not a quotient with a remainder.



The measurement. Ten laws, ten pictures, and not one of them shows a generator. That is the honest result: `∩`, `∪`, `°` and composition are what the Frobenius generators build, and `/` is none of those — it is posited, a field of `DivisionAllegory` with nothing to unfold. Eight of the ten carry the long-division bar instead, which is a metaphor for the universal property rather than a composite of anything, and the two that do not say why in place.

What is not drawn, and why. The cheatsheet's pointwise row, `x (T\U) y ⟺ ∀w. w T x ⟹ w U y`, quantifies over elements and has no arrow to draw. The rest of §8 is the negation half, and the first paragraphs of this section are the argument for leaving it out.